

# Ungear

---

## Ungear (*student proposal*)

---

**Assessment:** ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

**Proposal (as submitted):** *Ungear* is a **2D dungeon crawler that inverts progression** — instead of gaining gear, the hero **loses** it. Each floor cleared **breaks a piece of equipment**, and every loss changes how the run plays (when the lamp fails, the field of view shrinks, so the player must navigate the dark more carefully). Core loop: explore winding tunnels, scavenge potions left by fallen heroes, clear hordes of spiders, and push deeper as another piece of gear gives out. The cave doesn't change — the **player's toolkit shrinks**, forcing new tactics and decisions about when to spend saved potions; adaptation replaces raw power. Named load-bearing C++: a **real-time simulation loop holding a stable step with ≥200 live spiders**, via an **ECS** with contiguous component storage, a **capped spawn system**, **pathfinding that only runs when a wall blocks the way**, and **batched rendering**. Named technical features: a **robust real-time spawning system**; **enemy AI & pathfinding** (spiders crawl toward the hero by distance + line of sight, path resolved around walls); a **seeded random map generator** for paths + spawners; and a **swipe-based combat system**. Tools: **seeded procedural level generation** (specific seeds for a consistent experience across players); and an **enemy-behaviour tuning tool** (importer/exporter for speed, health, detection range, aggression — edit externally, import, export for balancing). Design features: the hero **grows weaker**, discarding equipment each stage; **≥3 "pictures-as-cutscene"** depicting equipment failure; **≥3 wave-designed stages + 1 boss stage**; and **≥3 spider types** with distinct behaviours. Team: **6 CS + 2 Design + 0 Art**.

*Note (module completeness):* the proposal doesn't name a **platform** — the swipe combat is touch-first, so this is planned as a mobile build (Pixel 9a per the module). It also doesn't mention an **External Database** or accounts, but the module mandates Firebase — plan to persist run records / seeds / best depth

there (a standard, small addition; see the External Database row + M1G12-style integration). Neither is a category concern.

*References:*

- [Vampire Survivors on Steam](#) (a horde-survival action roguelite where the screen fills with enemies and the player pushes to survive deeper — the hordes-of-enemies +  $\geq 200$ -live-agents performance precedent)
- [The Binding of Isaac: Rebirth on Steam](#) (a randomly-generated, run-based dungeon crawler where each run is a fresh combination of rooms and items — the run-based dungeon-crawler precedent)
- [Spelunky 2 on Steam](#) (procedurally-generated caves from seeds, producing unique but reproducible descents each playthrough — the seeded-procedural-cave + "push deeper" precedent)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

| Core Tech Requirement                                       | Ungeared |
|-------------------------------------------------------------|----------|
| External Database                                           | ✓        |
| Authentication »                                            | ●        |
| Analytics »                                                 | ●        |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓        |
| Application Communication                                   | ✓        |
| Continuous Integration                                      | ✓        |
| Data-Driven Architecture                                    | ✓        |
| Front-end + Back-end                                        | ✓        |
| Custom Tools Development                                    | ✓        |
| 2D Graphical Output                                         | ✓        |
| 3D (strong students only)                                   | ✗        |
| Networking System                                           | ●        |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — the swipe combat is touch-first; a Web-on-mobile (Emscripten→WASM) build is a viable alternative. The Native PC build is the development/tools vehicle (the enemy-behaviour tuning tool + the seed/level tooling). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for run records / seeds / best-depth persistence.

**Front end:** A 2D top-down dungeon renderer — the cave tunnels, the hero, up to  $\geq 200$  spiders (batched), potions/pickups, the shrinking field-of-view as

the lamp fails, the equipment/potion HUD and the "pictures-as-cutscape" equipment-failure screens. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

**Back end:** The real-time C++ simulation loop (ECS with contiguous storage, the capped spawning system, the wall-aware line-of-sight pathfinding that runs only when blocked, the equipment-degradation + potion economy, swipe-combat resolution and batched rendering), holding a stable step with  $\geq 200$  live spiders on Android; plus Firebase for run/seed persistence. **Languages:** C++ (engine); Firebase-backed services on the Extended tier.

**Backend scope:** Local now, server-ready (T2) — the run/seed-persistence seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — the ECS + contiguous component storage, the capped spawn system, wall-aware LOS pathfinding computed only when a wall blocks, batched rendering and the per-floor equipment-degradation logic together hold a stable step with  $\geq 200$  live spiders on the mobile target.
- *Data-driven:* spider types ( $\geq 3$ , with speed/health/detection-range/aggression), equipment pieces + their break-effects, potions, spawn tables, wave definitions and level seeds are all data — a new spider or gear effect is a data change (authored/tuned via the enemy-behaviour tool).
- *2D:* the top-down cave renderer, the batched spider horde, the shrinking-FOV lighting, the potion/gear HUD and the equipment-failure cutscene stills (custom-rendered). *3D:* not needed.
- *Custom tools:* two — the **seeded procedural level generator** (seeds  $\rightarrow$  reproducible maps with paths + scattered spawners) and the **enemy-behaviour tuning tool** (import/export spider stats + AI params for external balancing).
- *Networking:* single-player; run/seed persistence is async client $\leftrightarrow$ cloud (Application Communication ✓), so a Networking System is optional (●).
- *External DB:* run records, seeds and best depth (Firebase — add this per the module). *Auth* »: optional accounts. *Analytics* »: optional — per-floor deaths + potion usage for balancing.

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the ECS loop (capped spawns + the  $\geq 200$ -spider budget) + swipe input.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep C++ loop over an ECS with contiguous component storage, a capped real-time spawning system, and swipe-gesture input, decoupled from rendering; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the AI/pathfinding (M1G8) and rendering (M1G4) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** An ECS/entity model with spider types (speed/health/detection-range/aggression), equipment pieces + break-effects, potions, spawn tables, wave definitions and level seeds in hot-reloadable JSON — a new spider or gear effect is a data change.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (the enemy-behaviour tuning tool), M1G8 (AI data), M2G13 (the degradation/potion systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (entity/spawn counts, pathfinding triggers, FOV state), Tracy profiling of a  $\geq 200$ -spider stress test to prove the stable step on Android, Catch2 tests (deterministic seeded generation,

pathfinding-only-when-blocked, data loading), on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the  $\geq 200$ -spider budget M2G1 validates.

- **G4 2D Graphics** — the batched top-down renderer for the cave + the  $\geq 200$ -spider horde + shrinking FOV.

Tech & dependencies

- **Tech: Web build:** A sprite/tile renderer with batching for the many spiders, a camera, the potion/gear HUD, and a lamp/FOV lighting mask that shrinks as gear breaks; SDL2/3/OpenGL ES, GLSL, stb\_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2G4 (advanced/juice); every visible frame.

- **G7 Procedural Level Generation** — the **seeded** map generator (paths + scattered spawners, reproducible across players).

Tech & dependencies

- **Tech:** Seeded RNG + a cave/tunnel algorithm (BSP / cellular automata / drunkard's walk) placing paths and spawners, with validation tests that a seed always yields a completable layout; identical output for a given seed across devices.
- **Prerequisites:** M1G2 (level/seed data model).
- **Feeds into:** M2G14 (proc-gen II — stage/wave variety); the "consistent experience between players" requirement.

- **G8 Enemy AI & Pathfinding** — spiders crawl toward the hero by distance + line of sight; path resolved around walls, only when blocked.

Tech & dependencies

- **Tech:** Line-of-sight/distance seeking with A\* invoked only when a wall blocks the direct path (a cheap direct-chase fallback otherwise), steering to avoid clumping, tuned from data (custom C++); designed to stay cheap across  $\geq 200$  agents.
- **Prerequisites:** G1 (tick), G2 (spider data).
- **Feeds into:** M2G10 (advanced AI — the  $\geq 3$  spider behaviours + boss); the horde-combat pillar.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — the  $\geq 200$ -spider budget hardened + on-device validation.

Tech & dependencies

- **Tech: Web build:** Memory/component pools, cache-friendly ECS iteration, Tracy profiling of the  $\geq 200$ -spider stress case, ASan on MSVC, and on-device validation (Pixel 9a) holding the frame budget; the Firebase/Linux-server seam (run/seed persistence) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a stable step under the full horde.

- **G2 ★ Content Editor — Core (Enemy-Behaviour Tuning Tool)** — import/export spider stats + AI params for external balancing.

Tech & dependencies

- **Tech:** Dear ImGui (or a small external editor) exposing speed/health/detection-range/aggression per spider type with round-trip JSON import/export, live preview and undo/redo, so designers rebalance without recompiling.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (spider/AI data model).
- **Feeds into:** M2D3 (data-driven balance); the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (seeded-generation determinism, pathfinding-only-when-blocked, degradation logic, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).

- **Feeds into:** Release/showcase builds of every goal.
- **G10 AI — Advanced** — the  $\geq 3$  spider behaviours matured + the boss.  
Tech & dependencies
  - **Tech:** Behaviour trees / FSMs giving the  $\geq 3$  spider types distinct roles (rushers, ambushers, ranged/spitters, etc.), a boss behaviour, and difficulty scaling as gear breaks, all tuned from data.
  - **Prerequisites:** M1G8 (AI & pathfinding foundation).
  - **Feeds into:** The  $\geq 3$  wave stages + the boss stage; the "adapt as you weaken" pressure.
- **G13 Gameplay Systems II** — the **inverse-progression** system: equipment degradation + the potion economy.  
Tech & dependencies
  - **Tech:** C++ systems over the ECS for the per-floor equipment-break rules and their run-changing effects (lamp  $\rightarrow$  FOV shrink, etc.), the potion inventory + save/spend decision economy, and the weakening-difficulty curve; balancing-data-driven.
  - **Prerequisites:** M1G2 (equipment/potion data), M1G1 (loop).
  - **Feeds into:** The USP at showcase; T2 gear/effect expansion.
- **G14 Procedural Generation II** — stage/wave variety + pacing across the  $\geq 3$  stages + boss.  
Tech & dependencies
  - **Tech:** Constraint-based generation for the designed wave stages and the boss arena (spawner density, path length, pacing curves) layered on the seeded generator, with automated validation of completability.
  - **Prerequisites:** M1G7 (seeded generation).
  - **Feeds into:** Content variety at showcase; the  $\geq 3$  stages + boss.

## Track Goals & Team Composition

**Recommended composition:** 6 CS + 2 Design + 0 Art (8) — **as proposed**, and a good fit: with **no artist**, the two designers absorb the **D11 conditional** (Art POC / Art Implementation Quality) each milestone (this is exactly why a 0-artist team is advised to carry 2 designers), so one can own the game/systems design (the degradation economy, the  $\geq 3$  stages + boss, the  $\geq 3$  spider behaviours, the swipe game-feel) while the other carries the art-surface conditional and the narrative cutscenes — both tracks stay reachable at the full 4-goals-per-designer load. Plan a **geometric / stylised programmer-art** direction (readable cave tiles, colour-coded spider types, silhouette-first gear) that suits a 0-artist team; the  $\geq 3$  "pictures-as-cutscape" stills are a scoped, design-ownable art deliverable. *Scope note (advisory):* an ECS holding  $\geq 200$  agents + wall-aware LOS pathfinding + seeded proc-gen + the degradation system + two tools is a **strong, focused CS scope** — one seeded stage with  $\sim 1$  spider type, basic degradation and the horde at target count is the M1/M2 floor; the  $\geq 3$  stages, the boss, all  $\geq 3$  spider behaviours, the full gear-break table and the cutscenes are the stretch.

**Track goals** — suggested Design / UI-UX / TEAM picks for this project (no Art track — 0 artists, so the Art surface is carried by the Design-owned D11 conditional + the design-owned cutscene stills); deliverables & quality ladders live in the track pool documents. Full load = 4 goals per designer per milestone across Design + UI/UX (8 combined for 2 designers).

- **Design M1:**

- M1D1 ★ Features Design Document (the inverse-progression USP, the equipment-break-effects, the potion economy, the  $\geq 3$  stages + boss and the  $\geq 3$  spider types; primary benchmark e.g. Spelunky 2 / The Binding of Isaac, plus a Singapore benchmark)
- M1D2 Core Gameplay Loop Prototype (a greybox proving explore → clear horde → lose gear → adapt is fun) · prototype in Unity
- M1D4 Game Feel & Touch Interaction Specification (the swipe-combat feel — gesture zones, hit-stop, feedback timing — as tunable data)
- M1D6 Systems & Economy Design (the equipment-degradation source/sink, the potion save-or-spend economy and the spawn/wave interlocks — every system with a purpose)

- M1D5 Level & Progression Design (the  $\geq 3$  stages + boss, the difficulty curve that rises as the hero weakens)
- M1D11 Art Proof of Concept \*(conditional — mandatory for 0-artist teams; Design-owned)\* — the stylised/programmer-art visual direction: palette, cave/spider/gear readability, and the equipment-failure cutscene style; 2 mockups.

• **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D2 Working Prototype — Full-Content Build (the polished core: explore → horde → degrade → boss)
- M2D3 Systems Balance & Tuning — Data-Driven (degradation/potion/spider balance via the tuning tool's JSON, no code changes)
- M2D4 Game Feel & Feedback Pass ("Juice") (swipe-hit reactions, gear-break punctuation, horde-readable feedback)
- M2D7 Narrative Presentation & Delivery (the  $\geq 3$  equipment-failure "pictures-as-cutscene" beats integrated in the build)
- M2D11 Art Implementation Quality \*(conditional — mandatory for 0-artist teams; Design-owned)\* — all in-game art + the cutscene stills implemented in-engine, no placeholders.

• **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the gear/potion HUD, the shrinking-FOV readability and the depth/floor indicator, plus a wireframe baseline.
- M1U3 Accessibility & Readability Foundations — legibility under the shrinking field-of-view; contrast and touch-target sizing for the swipe combat.

• **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — menus, HUD, the gear-break screens and the cutscene flow navigable unaided on the mobile target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished swipe-combat + gear-loss UX tested with players on the Pixel 9a.

• **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (tense cave

BGM, spider skitter/attack, swipe-hit, potion-drink, gear-break and low-lamp cues), suitable and correctly triggered.

- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a busy horde.

• **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (cave BGM + spider / swipe-hit / potion / gear-break / boss / defeat SFX,  $\geq 6$  types), scaling with horde intensity.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct gear-break + boss cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the enemy-behaviour tuning tool + the seeded level generator), prototype demo, team post-mortem (wk14)

---

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*